

PROJECT BLUEPRINT: RETRO-WAVE HORROR DESIGN

Target Engine: Godot Engine (GDScript)
Aesthetic Paradigm: 1980s Outrun Neon mixed with Gritty VHS Analog Horror (Chilla's Art Style)

1. Executive Visual Vision

This document locks in the foundational guidelines for a high-atmosphere, budget-conscious 3D retro-wave horror title. The artwork blends high-contrast neon palettes with an aggressive low-fidelity lens overlay. It mirrors the pacing and mechanical tension found in Chilla's Art horror games, set against a cyberpunk backdrop.

2. Technical Architecture & Repository Layout

To optimize workspace stability and accommodate heavy 3D assets on a strict budget, development relies on a twin GitHub repository architecture managed via Git LFS pointers:

- **Code / Logic Repo:** Contains lightweight text data, code scripts (GDScript), scenes, material configurations, and UI logic.
- **Asset Drop Zone Repo:** Acts as an isolated cloud ingestion folder. Raw binary files (.zip, .fbx, .hdr, .png, .wav) land here, governed by an internal .gitattributes mapping to keep main repositories clean.

3. Budget-Optimized Model Delegation Profile

The technical workload is strategically split among processing models to protect compute budgets while guaranteeing deep-system execution accuracy:

Task Category	Assigned System Model	Execution Execution Method
Architecture Blueprinting	Opus 5.1 (Premium Mode)	Generates mathematical master shaders, custom interaction frameworks, core save loops, and complex dialogue trees.
Asset Sorting & Vetting	Sol (Assistant Mode)	Unzips raw packages inside Drop Zone, sanitizes file structures to snake_case, and shifts contents into sorted pipelines.
Logic Script Execution	Sol (Assistant Mode)	Compiles standard mechanics: player physics, footstep triggers, flashlight batteries, raycast item pick-ups, and bug fixes.

4. Mandated Folder Tree Rules

AI extensions executing file writes locally or inside staging paths must enforce this structural map:

- Textures/ — Neon arrays, glowing signage, rusty metal finishes, concrete tiling overlays.
- Models/ — Low-poly asset meshes, vehicle frames, structural walls, interactable environment items.
- Shaders/ — CRT lens post-processing scripts, chromatic distortion math, custom screen glitches.
- Audio/ — Low hums, static white-noise frequencies, synthetic bass loops, physical step impacts.
- HDRI/ — 360-degree high-dynamic range panoramic environments for horizon space reflection lighting.

5. Foundational CRT/VHS Shader Formula

The visual landscape depends on a standard canvas distortion rendering layer. The production shader formula requires immediate configuration parameters inside Godot for **tracking jitter**, **horizontal micro waves**, **chromatic aberration color offsets**, **horizontal CRT scanlines**, **a round bubble lens vignette**, and a layer of **procedural grain noise** to mimic magnetic tracking degradation.

This is for informational purposes only. For medical advice or diagnosis, consult a professional. AI responses may include mistakes.